

able to hit the next character for at least 10 milliseconds.

Modular RTOS

As far as embedded systems go, the requirements vary from application to application. A simple gadget may require a small and basic system, while mission-critical, advanced equipment may require more flexible and faster system. Earlier, developers would just choose between a 'light' RTOS or an 'advanced' one, depending on their requirements. But today, the choice is not that simple.

"While minimal configurations are popular for some applications, requirements of many applications are getting more complex. Developers who write their own RTOS or use a minimal one, end up writing more codes to fill the gap between their application needs and what the RTOS offers. This consumes more time and increases the engineering costs," says Brad Dixon, senior product marketing manager, Mentor Graphics.

Nowadays, companies are beginning to provide modular real-time operating systems, enabling developers to choose the parts that they require. Nucleus RTOS, for example, gives developers the freedom of choice. From small applications using minimal kernel and core services to more advanced applications utilising available middleware, Nucleus RTOS satisfies a wide range of customer requirements. The Nucleus RTOS is hardware independent and has been ported to a number of ARM, Power and MIPS architecture central processing units (CPUs).

Mentor offers a spectrum of software to address many types of development needs. In a minimal configuration, the core Nucleus RTOS services are very few. Nucleus middleware components can be included to provide additional services such as USB, networking and graphics. For applications that require more extensive system services, Mentor can bring Linux and Android technology to bear on these projects either independently or in

Real-time operating systems—A quick intro

If you have five tabs opened on your browser, then it is likely that the system will take longer than expected to open a webpage on a sixth tab. When you have a dozen services running on your computer, it is really difficult to say how long the next program might take to load and operate. That is one of the inherent problems in general-purpose computers and their operating systems—you do not know how long it might take to complete an instruction, since it is a function of several other factors such as the number of other services already running, resource utilisation by the other applications, etc.

Just imagine if the case were the same with the operating system running on a computer embedded in, say, a missile launcher or a robotic arm in an automotive assembly line! There will be numerous other machinery and processes dependent on these time-critical equipment or gadgets, and you really need to have an accurate estimate of how long the instrument might take to do its work. This is where a real-time operating system comes into play—it essentially ensures 'deterministic' timing, that is, the amount of time taken by the operating system to respond to an instruction is constant (and small), irrespective of the load, thereby enabling it to be used in real-time applications.

Real-time operating systems (RTOS) are different from the general-purpose operating systems running on our personal computers. Mainly, they are fast; the response times are highly predictable; they are small and occupy very little memory; they consume fewer resources, ensuring that these resources are available for the other operations of the device; they include significantly fewer functions—that is, only those relevant for the device; and are sometimes architecture or microprocessor-dependent.

conjunction with the Nucleus RTOS," explains Dixon.

Modular RTOS can serve diverse applications, ranging from general-purpose and household appliances to process control and digital signal processing.

Linux-based RTOS

When speaking about modularity, customisability, 'light' versions, choosing the right RTOS, etc, Linux-based RTOS definitely deserves a mention. Of late, quite a few Linux-based real-time operating systems have come up. Some of them are: Stable Hybrid Release (SHR, for the OpenMoko-based mobile phones), Red Hat Embedded Linux and eCos.

The SHR is currently undergoing an extensive development, and at the moment, there are two versions—SHR-Lite and SHR-Full—that only differ in the number of default applications installed. Soon, an SHR-terminal will be developed that ships only with the core basics, not even X11 or a Graphical Interface. It will be used by people who want to create their own RTOS distribution on top of SHR, or use it for other purposes that do not need an X server—from using it for fun ap-

plications like connecting the device to a robot controlled by USB and controlling the robot using SMS/phone calls, to using it as a core location-aware (GPS) and always-on (GSM) computer for various purposes.

"In the open source world, many times, less is more, as users usually like to pick their own software and customise the product as they like, and not as the developers like," say Julien Cassagnol, Tom Hachoen, Sebastian Krzyszowski and Klaus Kurzmann, the core team of SHR.

"We prefer using simpler versions of applications for better performance (speed) and more battery time, as in cellphones, battery time is the most important feature. And, all our source code is available for download and customisation. Users also play an important role in designing our core user interface and applications through their feedback (wikis, mailing lists, Internet relay chat, etc)," they add. User contribution is one of the main benefits of Linux-based open source RTOS; it means that talented people from across the world are working on the software, and hence improving the quality and adding more features.